

CS2300: Intro to Database Systems & File Structures

Lecture Notes

Functional Dependency Preservation & Non-additive Join Property in
Normalizaion

Missouri S&T

Instructor: San Yeung

[Spring Semester 2025]

1 Functional Dependency Preservation

During the process of **normalization**, a decomposition of a relation schema R into $R_1, R_2, \dots, R_n$ is said to **preserve functional dependencies** if the union of the FDs in the new, projected schemas is **equivalent to the original FDs** in R . That is, the original FDs can still be enforced or obtained **without needing to join the decomposed relations**.

Formally, if F is the set of functional dependencies on R , and F_i is the projection of F onto R_i :

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

1.1 Example 1: FD Preservation

This is an example of a decomposition into **Third Normal Form (3NF)**, where all functional dependencies are preserved.

Let $R(\underline{A}, B, C)$ with functional dependencies:

$$F = \{A \rightarrow B, B \rightarrow C\}$$

Decompose R into:

$$R_1(\underline{A}, B), \quad R_2(\underline{B}, C)$$

Projected FDs:

$$F_1 = \{A \rightarrow B\}, \quad F_2 = \{B \rightarrow C\}$$

Together:

$$F_1 \cup F_2 = \{A \rightarrow B, B \rightarrow C\} = F^+ \Rightarrow \text{FDs are preserved}$$

1.2 Example 2: FD Not Preserved

This is an example of a decomposition into **Boyce-Codd Normal Form (BCNF)**, where not all functional dependencies are preserved.

Let $R(A, B, C)$ with functional dependencies:

$$F = \{AB \rightarrow C, AC \rightarrow B, C \rightarrow B\}$$

Decompose R into:

$$R_1(\underline{A}, C), \quad R_2(\underline{C}, B)$$

Projected FDs:

$$F_1 = \{AC \rightarrow AC\}(\text{trivial}), \quad F_2 = \{C \rightarrow B\}$$

Together:

$$F_1 \cup F_2 = \{AC \rightarrow AC, C \rightarrow B\} \neq F^+ \Rightarrow \text{FDs are not preserved}$$

because $AB \rightarrow C$ and $AC \rightarrow B$ are not present in $F_1 \cup F_2$.

1.3 Remarks

- FD Preservation ensures that integrity constraints are still enforceable on the decomposed tables.
- FD Preservation allows the DBMS to enforce dependencies without expensive joins, which makes normalized schemas practical for implementation.
- Not all decompositions preserve FDs—particularly when moving to higher normal forms like BCNF.
- Use FD-preserving decompositions when integrity enforcement is a priority.

2 Non-Additive (Lossless) Join Property

A decomposition of relation R into R_1 and R_2 is said to have the **non-additive (or lossless) join property** if we can recover the original relation R by performing a **natural join**:

$$R = R_1 \bowtie R_2$$

This property prevents spurious (incorrect) tuples from appearing in the result of the join. **A decomposition that does not have the non-additive join property is considered invalid.**

2.1 Testing for Non-Additive Join

Suppose a relation schema R is decomposed into R_1 and R_2 . The decomposition is said to be **lossless** (also called **non-additive**) with respect to the set of functional dependencies F in R **if and only if** the **natural join** of R_1 and R_2 yields exactly the original relation R , **without introducing spurious tuples**.

2.1.1 Testing for Lossless-Join using Functional Dependencies

We can **test** whether the decomposition is lossless using a **functional dependency-based test**:

A decomposition of R into R_1 and R_2 is **lossless** with respect to F if:

$$(R_1 \cap R_2) \rightarrow (R_1 - R_2) \quad \text{or} \quad (R_1 \cap R_2) \rightarrow (R_2 - R_1) \quad \text{holds in } F^+$$

What this means:

- The attributes common to both R_1 and R_2 (i.e., the join attributes) must form a valid functional dependency — either originally in F or implied in F^+ — that determines all attributes of either R_1 or R_2 .
- This ensures that the common attributes act as a **valid constraint** when the two tables are joined back together, preventing the creation of spurious or invalid tuples.

2.2 Example: Lossless Join

Let $R(A, B, C)$ with functional dependencies:

$$F = \{AB \rightarrow C, AC \rightarrow B, C \rightarrow B\}$$

Decompose R into:

$$R_1(\underline{A}, \underline{C}), \quad R_2(\underline{C}, \underline{B})$$

Test for Lossless Join:

The common attribute is C . We check if $C \rightarrow (R_1 - R_2)$ or $C \rightarrow (R_2 - R_1)$ holds in F^+ . Since $C \rightarrow B$ is in F , the lossless-Join condition is satisfied. However, not all FDs (e.g., $AC \rightarrow B$) are preserved in the decomposed schema.

2.2.1 Sample Data Illustration

Let the original relation R have the following tuples:

A	B	C
a1	b1	c1
a1	b2	c2
a2	b1	c3

Decomposing into:

- $R_1(A, C)$:

A	C
a1	c1
a1	c2
a2	c3

- $R_2(C, B)$:

C	B
c1	b1
c2	b2
c3	b1

Now perform the natural join between R_1 and R_2 on C :

A	B	C
a1	b1	c1
a1	b2	c2
a2	b1	c3

This matches the original relation — so the decomposition is **lossless**.

2.3 Example: Lossy Join (Additive Join)

Let $R(A, B, C)$ with functional dependencies:

$$F = \{AB \rightarrow C, AC \rightarrow B, C \rightarrow B\}$$

Decompose R into:

$$R_1(\underline{A}, B), \quad R_2(\underline{C}, B)$$

Test for Lossless Join:

The common attribute is B . We check if $B \rightarrow (R_1 - R_2)$ or $B \rightarrow (R_2 - R_1)$ holds in F^+ . Since $B \rightarrow A$ or $B \rightarrow C$ is not in F^+ , the lossless-Join condition is not satisfied.

2.3.1 Sample Data Illustration

Let the original relation R have the following tuples:

A	B	C
a1	b1	c1
a1	b2	c2
a2	b1	c3

Decomposing into:

- $R_1(A, B)$:

A	B
a1	b1
a1	b2
a2	b1

- $R_2(C, B)$:

C	B
c1	b1
c2	b2
c3	b1

Now perform the natural join between R_1 and R_2 on B :

A	B	C
a1	b1	c1
a1	b1	c3
a1	b2	c2
a2	b1	c1
a2	b1	c3

This does not match the original relation — so the decomposition is **lossy**, or additive.